

אנטי-ניפוי באגים: דגלי ניפוי באגים

(<https://anti-debug.checkpoint.com>) חזרה לעמוד הראשי

תוכן

דגלי ניפוי באגים

- 1. שימוש ב-API של Win32
 - 1.1. IsDebuggerPresent()
 - 1.2. CheckRemoteDebuggerPresent()
 - 1.3. תהליך מידע של NtQuery()
 - 1.3.1. יציאת תהליך ניפוי באגים
 - 1.3.2. דגלי ניפוי באגים של תהליך
 - 1.3.3. זיהוי אובייקט של תהליך ניפוי באגים
 - 1.4. מידע על ערימת תהליך RtlQuery()
 - 1.5. RtlQueryProcessDebugInformation()
 - 1.6. מידע מערכת NtQuery()
 - הקלות
- 2. בדיקות ידניות
 - 2.1. דגל PEB!Beingbugged
 - 2.2. דגל NtGlobalFlag
 - 2.3. דגלי ערימה
 - 2.4. הגנה מפני ערימות
 - 2.5. בדיקת מבנה KUSER_SHARED_DATA
 - הקלות

דגלי ניפוי באגים

דגלים מיוחדים בטבלאות מערכת, הנמצאים בזיכרון התהליך ואשר מערכת הפעלה מגדירה, יכולים לשמש כדי לציין שהתהליך נמצא תחת ניפוי באגים. ניתן לאמת את מצבי הדגלים הללו באמצעות פונקציות API ספציפיות או על ידי בחינת טבלאות המערכת בזיכרון.

טכניקות אלו הן הנפוצות ביותר בשימוש על ידי תוכנות זדוניות.

1. שימוש ב-API של Win32

הטכניקות הבאות משתמשות בפונקציות API קיימות (WinAPI או NativeAPI) שבודקות מבני מערכת בזיכרון התהליך עבור דגלים מסוימים המציינים שהתהליך נמצא כעת בתהליך תיקון באגים.

1.1. IsDebuggerPresent()

קובעת האם התהליך הנוכחי נמצא תחת ניפוי באגים על ידי ניפוי באגים במצב משתמש כגון `kernel32!IsDebuggerPresent()` הפונקציה בודקת רק את הדגל `OlllyDbgBeingDebugged` או `x64dbg`. באופן כללי, הפונקציה בודקת רק את הדגל (PEB). (https://www.nirsoft.net/kernel_struct/vista/PEB.html)

ניתן להשתמש בקוד הבא כדי לסיים תהליך אם הוא נמצא בתהליך ניפוי באגים:

קוד הרכבה

```

    call IsDebuggerPresent
    test al, al
    jne being_debugged
    ...
being_debugged:
    push 1
    call ExitProcess

```

קוד ++C/C

```

if (IsDebuggerPresent())
    ExitProcess(-1);

```

1.2. CheckRemoteDebuggerPresent()

בודקת אם ניפוי באגים (בתהליך אחר באותה מכונה) מחובר לתהליך הנוכחי. `kernel32!CheckRemoteDebuggerPresent()` הפונקציה

קוד ++C/C

```

BOOL bDebuggerPresent;
if (TRUE == CheckRemoteDebuggerPresent(GetCurrentProcess(), &bDebuggerPresent) &&
    TRUE == bDebuggerPresent)
    ExitProcess(-1);

```

הרכבה x86

```

    lea eax, [bDebuggerPresent]
    push eax
    push -1 ; GetCurrentProcess()
    call CheckRemoteDebuggerPresent
    cmp [bDebuggerPresent], 1
    jz being_debugged
    ...
being_debugged:
    push -1
    call ExitProcess

```

מכלול x86-64

```

    lea rdx, [bDebuggerPresent]
    mov rcx, -1 ; GetCurrentProcess()
    call CheckRemoteDebuggerPresent
    cmp [bDebuggerPresent], 1
    jz being_debugged
    ...
being_debugged:
    mov ecx, -1
    call ExitProcess

```

1.3. תהליך מידע של NtQuery()

יכולה לאחר סוג אחר של מידע מתהליך. היא מקבלת פרמטר `ntdll!NtQueryInformationProcess()` הפונקציה `ProcessInformationClass` אשר מציין את המידע שברצונך לקבל ומגדיר את סוג הפלט של הפרמטר `ProcessInformationClass`.

1.3.1. יציאת תהליך ניפוי באגים

. קיימת מחלקה `ntdll!NtQueryInformationProcess()` לחזור את מספר הפורט של ניפוי הבאגים עבור התהליך באמצעות הפונקציה מתועדת (<https://docs.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntqueryinformationprocess#PROCESSDEBUGPORT>) `ProcessDebugPort`, אשר מאחזרת ערך `DWORD` השווה ל-`0xFFFFFFFF` אם התהליך נמצא תחת ניפוי באגים.1-(עשרוני)

קוד C/C++

```
typedef NTSTATUS (NTAPI *TntQueryInformationProcess)(
    IN HANDLE          ProcessHandle,
    IN PROCESSINFOCLASS ProcessInformationClass,
    OUT PVOID           ProcessInformation,
    IN ULONG            ProcessInformationLength,
    OUT PULONG          ReturnLength
);

HMODULE hNtdll = LoadLibraryA("ntdll.dll");
if (hNtdll)
{
    auto pfnNtQueryInformationProcess = (TntQueryInformationProcess)GetProcAddress(
        hNtdll, "NtQueryInformationProcess");

    if (pfnNtQueryInformationProcess)
    {
        DWORD dwProcessDebugPort, dwReturned;
        NTSTATUS status = pfnNtQueryInformationProcess(
            GetCurrentProcess(),
            ProcessDebugPort,
            &dwProcessDebugPort,
            sizeof(DWORD),
            &dwReturned);

        if (NT_SUCCESS(status) && (-1 == dwProcessDebugPort))
            ExitProcess(-1);
    }
}
```

הרכבה x86

```
leax eax, [dwReturned]
push eax ; ReturnLength
push 4 ; ProcessInformationLength
leax ecx, [dwProcessDebugPort]
push ecx ; ProcessInformation
push 7 ; ProcessInformationClass
push -1 ; ProcessHandle
call NtQueryInformationProcess
inc dword ptr [dwProcessDebugPort]
jz being_debugged
...
being_debugged:
push -1
call ExitProcess
```

מכלול x86-64

```

lea rcx, [dwReturned]
push rcx ; ReturnLength
mov r9d, 4 ; ProcessInformationLength
lea r8, [dwProcessDebugPort]
; ProcessInformation
mov edx, 7 ; ProcessInformationClass
mov rcx, -1 ; ProcessHandle
call NtQueryInformationProcess
cmp dword ptr [dwProcessDebugPort], -1
jz being_debugged
...
being_debugged:
mov ecx, -1
call ExitProcess

```

1.3.2. דגלי ניפוי באגים של תהליך

, המייצג אובייקט תהליך, מכיל(EPROCESS (https://www.nirsoft.net/kernel_struct/vista/EPROCESS.html) מבינה ליבה בשם NoDebugInherit את השדה ProcessDebugFlags (0x1f.(ניתן לאחזר את הערך ההפוך של שדה זה באמצעות מחלקה לא מתועדת את השדה , קיים ניפוי שגיאות.0לכן, אם הערך המוחזר הוא

קוד ++C/C

```

typedef NTSTATUS(NTAPI *TntQueryInformationProcess)(
    IN HANDLE          ProcessHandle,
    IN DWORD            ProcessInformationClass,
    OUT PVOID           ProcessInformation,
    IN ULONG            ProcessInformationLength,
    OUT PULONG          ReturnLength
);

HMODULE hNtdll = LoadLibraryA("ntdll.dll");
if (hNtdll)
{
    auto pfnNtQueryInformationProcess = (TntQueryInformationProcess)GetProcAddress(
        hNtdll, "NtQueryInformationProcess");

    if (pfnNtQueryInformationProcess)
    {
        DWORD dwProcessDebugFlags, dwReturned;
        const DWORD ProcessDebugFlags = 0x1f;
        NTSTATUS status = pfnNtQueryInformationProcess(
            GetCurrentProcess(),
            ProcessDebugFlags,
            &dwProcessDebugFlags,
            sizeof(DWORD),
            &dwReturned);

        if (NT_SUCCESS(status) && (0 == dwProcessDebugFlags))
            ExitProcess(-1);
    }
}

```

הרכבה x86

```

lea eax, [dwReturned]
push eax ; ReturnLength
push 4 ; ProcessInformationLength
lea ecx, [dwProcessDebugFlags]
push ecx ; ProcessInformation
push 1Fh ; ProcessInformationClass
push -1 ; ProcessHandle
call NtQueryInformationProcess
cmp dword ptr [dwProcessDebugFlags], 0
jz being_debugged
...
being_debugged:
push -1
call ExitProcess

```

מכלול x86-64

```

lea rcx, [dwReturned]
push rcx ; ReturnLength
mov r9d, 4 ; ProcessInformationLength
lea r8, [dwProcessDebugFlags]
; ProcessInformation
mov edx, 1Fh ; ProcessInformationClass
mov rcx, -1 ; ProcessHandle
call NtQueryInformationProcess
cmp dword ptr [dwProcessDebugFlags], 0
jz being_debugged
...
being_debugged:
mov ecx, -1
call ExitProcess

```

1.3.3. זיהוי אובייקט של תהליךניפוי באגים

When debugging begins, a kernel object called “debug object” is created. It is possible to query for the value of this handle by using the undocumented `ProcessDebugObjectHandle` (0x1e) class.

C/C++ Code

```

typedef NTSTATUS(NTAPI * TntQueryInformationProcess)(
    IN HANDLE          ProcessHandle,
    IN DWORD           ProcessInformationClass,
    OUT PVOID          ProcessInformation,
    IN ULONG           ProcessInformationLength,
    OUT PULONG         ReturnLength
);

HMODULE hNtdll = LoadLibraryA("ntdll.dll");
if (hNtdll)
{
    auto pfnNtQueryInformationProcess = (TntQueryInformationProcess)GetProcAddress(
        hNtdll, "NtQueryInformationProcess");

    if (pfnNtQueryInformationProcess)
    {
        DWORD dwReturned;
        HANDLE hProcessDebugObject = 0;
        const DWORD ProcessDebugObjectHandle = 0x1e;
        NTSTATUS status = pfnNtQueryInformationProcess(
            GetCurrentProcess(),
            ProcessDebugObjectHandle,
            &hProcessDebugObject,
            sizeof(HANDLE),
            &dwReturned);

        if (NT_SUCCESS(status) && (0 != hProcessDebugObject))
            ExitProcess(-1);
    }
}

```

x86 Assembly

```

    lea eax, [dwReturned]
    push eax ; ReturnLength
    push 4   ; ProcessInformationLength
    lea ecx, [hProcessDebugObject]
    push ecx ; ProcessInformation
    push 1Eh ; ProcessInformationClass
    push -1  ; ProcessHandle
    call NtQueryInformationProcess
    cmp dword ptr [hProcessDebugObject], 0
    jnz being_debugged
    ...
being_debugged:
    push -1
    call ExitProcess

```

x86-64 Assembly

```

    lea rcx, [dwReturned]
    push rcx      ; ReturnLength
    mov r9d, 4    ; ProcessInformationLength
    lea r8, [hProcessDebugObject]
                ; ProcessInformation
    mov edx, 1Eh ; ProcessInformationClass
    mov rcx, -1  ; ProcessHandle
    call NtQueryInformationProcess
    cmp dword ptr [hProcessDebugObject], 0
    jnz being_debugged
    ...
being_debugged:
    mov ecx, -1
    call ExitProcess

```

1.4. RtlQueryProcessHeapInformation()

The `ntdll!RtlQueryProcessHeapInformation()` function can be used to read the heap flags from the process memory of the current process.

C/C++ Code

```

bool Check()
{
    ntdll::PDEBUG_BUFFER pDebugBuffer = ntdll::RtlCreateQueryDebugBuffer(0, FALSE);
    if (!SUCCEEDED(ntdll::RtlQueryProcessHeapInformation((ntdll::PRTL_DEBUG_INFORMATION)pDebugBuffer)))
        return false;

    ULONG dwFlags = ((ntdll::PRTL_PROCESS_HEAPS)pDebugBuffer->HeapInformation)->Heaps[0].Flags;
    return dwFlags & ~HEAP_GROWABLE;
}

```

1.5. RtlQueryProcessDebugInformation()

The `ntdll!RtlQueryProcessDebugInformation()` function can be used to read certain fields from the process memory of the requested process, including the heap flags.

C/C++ Code

```

bool Check()
{
    ntdll::PDEBUG_BUFFER pDebugBuffer = ntdll::RtlCreateQueryDebugBuffer(0, FALSE);
    if (!SUCCEEDED(ntdll::RtlQueryProcessDebugInformation(GetCurrentProcessId(), ntdll::PDI_HEAPS | ntdll::PDI_HEAP_BLOCKS, pDebugBuffer)))
        return false;

    ULONG dwFlags = ((ntdll::PRTL_PROCESS_HEAPS)pDebugBuffer->HeapInformation)->Heaps[0].Flags;
    return dwFlags & ~HEAP_GROWABLE;
}

```

1.6. NtQuerySystemInformation()

The `ntdll!NtQuerySystemInformation()` function accepts a parameter which is the class of information to query. Most of the classes are not documented. This includes the `SystemKernelDebuggerInformation` (0x23) class, which has existed since Windows NT. The

SystemKernelDebuggerInformation class returns the value of two flags: KdDebuggerEnabled in al, and KdDebuggerNotPresent in ah. Therefore, the return value in ah is zero if a kernel debugger is present.

C/C++ Code

```
enum { SystemKernelDebuggerInformation = 0x23 };

typedef struct _SYSTEM_KERNEL_DEBUGGER_INFORMATION {
    BOOLEAN DebuggerEnabled;
    BOOLEAN DebuggerNotPresent;
} SYSTEM_KERNEL_DEBUGGER_INFORMATION, *PSYSTEM_KERNEL_DEBUGGER_INFORMATION;

bool Check()
{
    NTSTATUS status;
    SYSTEM_KERNEL_DEBUGGER_INFORMATION SystemInfo;

    status = NtQuerySystemInformation(
        (SYSTEM_INFORMATION_CLASS)SystemKernelDebuggerInformation,
        &SystemInfo,
        sizeof(SystemInfo),
        NULL);

    return SUCCEEDED(status)
        ? (SystemInfo.DebuggerEnabled && !SystemInfo.DebuggerNotPresent)
        : false;
}
```

Mitigations

- For IsDebuggerPresent(): Set the BeingDebugged flag of the Process Environment Block (PEB) to 0. See [BeingDebugged Flag Mitigation](#) for further information.
- For CheckRemoteDebuggerPresent() and NtQueryInformationProcess():
As CheckRemoteDebuggerPresent() calls NtQueryInformationProcess(), the only way is to hook the NtQueryInformationProcess() and set the following values in return buffers:
 - 0 (or any value except -1) in case of a ProcessDebugPort query.
 - Non-zero value in case of a ProcessDebugFlags query.
 - 0 in case of a ProcessDebugObjectHandle query.
- The only way to mitigate these checks with RtlQueryProcessHeapInformation(), RtlQueryProcessDebugInformation() and NtQuerySystemInformation() functions is to hook them and modify the returned values:
 - RTL_PROCESS_HEAPS::HeapInformation::Heaps[0]::Flags to HEAP_GROWABLE for RtlQueryProcessHeapInformation() and RtlQueryProcessDebugInformation().
 - SYSTEM_KERNEL_DEBUGGER_INFORMATION::DebuggerEnabled to 0 and SYSTEM_KERNEL_DEBUGGER_INFORMATION::DebuggerNotPresent to 1 for the NtQuerySystemInformation() function in case of a SystemKernelDebuggerInformation query.

2. Manual checks

The following approaches are used to validate debugging flags in system structures. They examine the process memory manually without using special debug API functions.

2.1. PEB!BeingDebugged Flag

This method is just another way to check BeingDebugged flag of PEB without calling IsDebuggerPresent().

32Bit Process

```
mov eax, fs:[30h]
cmp byte ptr [eax+2], 0
jne being_debugged
```

64Bit Process

```
mov rax, gs:[60h]
cmp byte ptr [rax+2], 0
jne being_debugged
```

WOW64 Process

```
mov eax, fs:[30h]
cmp byte ptr [eax+1002h], 0
```

C/C++ Code

```
#ifndef _WIN64
PPEB pPeb = (PPEB)__readfsdword(0x30);
#else
PPEB pPeb = (PPEB)__readgsqword(0x60);
#endif // _WIN64

if (pPeb->BeingDebugged)
    goto being_debugged;
```

2.2. NtGlobalFlag

The NtGlobalFlag field of the Process Environment Block (0x68 offset on 32-Bit and 0xBC on 64-bit Windows) is 0 by default. Attaching a debugger doesn't change the value of NtGlobalFlag. However, if the process was created by a debugger, the following flags will be set:

- FLG_HEAP_ENABLE_TAIL_CHECK (0x10)
- FLG_HEAP_ENABLE_FREE_CHECK (0x20)
- FLG_HEAP_VALIDATE_PARAMETERS (0x40)

The presence of a debugger can be detected by checking a combination of those flags.

32Bit Process

```
mov eax, fs:[30h]
mov al, [eax+68h]
and al, 70h
cmp al, 70h
jz being_debugged
```

64Bit Process

```

mov rax, gs:[60h]
mov al, [rax+BCh]
and al, 70h
cmp al, 70h
jz being_debugged

```

WOW64 Process

```

mov eax, fs:[30h]
mov al, [eax+10BCh]
and al, 70h
cmp al, 70h
jz being_debugged

```

C/C++ Code

```

#define FLG_HEAP_ENABLE_TAIL_CHECK    0x10
#define FLG_HEAP_ENABLE_FREE_CHECK   0x20
#define FLG_HEAP_VALIDATE_PARAMETERS 0x40
#define NT_GLOBAL_FLAG_DEBUGGED (FLG_HEAP_ENABLE_TAIL_CHECK | FLG_HEAP_ENABLE_FREE_CHECK | FLG_HEAP_VALIDATE_PARAMETERS)

#ifdef _WIN64
PPEB pPeb = (PPEB)__readfsdword(0x30);
DWORD dwNtGlobalFlag = *(PDWORD)((PBYTE)pPeb + 0x68);
#else
PPEB pPeb = (PPEB)__readgsqword(0x60);
DWORD dwNtGlobalFlag = *(PDWORD)((PBYTE)pPeb + 0xBC);
#endif // _WIN64

if (dwNtGlobalFlag & NT_GLOBAL_FLAG_DEBUGGED)
    goto being_debugged;

```

2.3. Heap Flags

The heap contains two fields which are affected by the presence of a debugger. Exactly how they are affected depends on the Windows version. These fields are **Flags** and **ForceFlags**.

The values of **Flags** and **ForceFlags** are normally set to **HEAP_GROWABLE** and **0**, respectively.

When a debugger is present, the **Flags** field is set to a combination of these flags on Windows NT, Windows 2000, and 32-bit Windows XP:

- **HEAP_GROWABLE** (2)
- **HEAP_TAIL_CHECKING_ENABLED** (0x20)
- **HEAP_FREE_CHECKING_ENABLED** (0x40)
- **HEAP_SKIP_VALIDATION_CHECKS** (0x10000000)
- **HEAP_VALIDATE_PARAMETERS_ENABLED** (0x40000000)

On 64-bit Windows XP, and Windows Vista and higher, if a debugger is present, the **Flags** field is set to a combination of these flags:

- **HEAP_GROWABLE** (2)
- **HEAP_TAIL_CHECKING_ENABLED** (0x20)
- **HEAP_FREE_CHECKING_ENABLED** (0x40)
- **HEAP_VALIDATE_PARAMETERS_ENABLED** (0x40000000)

When a debugger is present, the **ForceFlags** field is set to a combination of these flags:

- HEAP_TAIL_CHECKING_ENABLED (0x20)
- HEAP_FREE_CHECKING_ENABLED (0x40)
- HEAP_VALIDATE_PARAMETERS_ENABLED (0x40000000)

C/C++ Code

```
bool Check()
{
#ifdef _WIN64
    PPEB pPeb = (PPEB)__readfsdword(0x30);
    PVOID pHeapBase = !m_bIsWow64
        ? (PVOID)(*(PDWORD_PTR)((PBYTE)pPeb + 0x18))
        : (PVOID)(*(PDWORD_PTR)((PBYTE)pPeb + 0x1030));
    DWORD dwHeapFlagsOffset = IsWindowsVistaOrGreater()
        ? 0x40
        : 0x0C;
    DWORD dwHeapForceFlagsOffset = IsWindowsVistaOrGreater()
        ? 0x44
        : 0x10;
#else
    PPEB pPeb = (PPEB)__readgsqword(0x60);
    PVOID pHeapBase = (PVOID)(*(PDWORD_PTR)((PBYTE)pPeb + 0x30));
    DWORD dwHeapFlagsOffset = IsWindowsVistaOrGreater()
        ? 0x70
        : 0x14;
    DWORD dwHeapForceFlagsOffset = IsWindowsVistaOrGreater()
        ? 0x74
        : 0x18;
#endif // _WIN64

    PDWORD pdwHeapFlags = (PDWORD)((PBYTE)pHeapBase + dwHeapFlagsOffset);
    PDWORD pdwHeapForceFlags = (PDWORD)((PBYTE)pHeapBase + dwHeapForceFlagsOffset);
    return (*pdwHeapFlags & ~HEAP_GROWABLE) || (*pdwHeapForceFlags != 0);
}
```

2.4. Heap Protection

If the HEAP_TAIL_CHECKING_ENABLED flag is set in NtGlobalFlag, the sequence 0xABABABAB will be appended (twice in 32-Bit and 4 times in 64-Bit Windows) at the end of the allocated heap block.

יתווסף אם נדרשים בתים נוספים כדי למלא 0xFEEEEEEE, הרצף NtGlobalFlag מוגדר ב- HEAP_FREE_CHECKING_ENABLED אם הדגל. את החלל הריק עד לבלוק הזיכרון הבא.

קוד ++C/C

```
bool Check()
{
    PROCESS_HEAP_ENTRY HeapEntry = { 0 };
    do
    {
        if (!HeapWalk(GetProcessHeap(), &HeapEntry))
            return false;
    } while (HeapEntry.wFlags != PROCESS_HEAP_ENTRY_BUSY);

    PVOID pOverlapped = (PBYTE)HeapEntry.lpData + HeapEntry.cbData;
    return ((DWORD)(*(PDWORD)pOverlapped) == 0xABABABAB);
}
```

2.5. KUSER_SHARED_DATA בדיקת מבנה

, מנהל התקן ליבה שמטרתו (https://github.com/mrexodia/TitanHide/issues/18) עבור TitanHide טכניקה זו תוארה במקור כבעיה כאן והשדות שלו זמין KUSER_SHARED_DATA להסתיר ניפוי באגים מגילוי. התייעוד המפורט עבור המבנה (https://www.geoffchappell.com/studies/windows/km/ntoskrnl/inc/api/ntexapi_x/kuser_shared_data/index.htm).

0x7ffe0000 + 0x2d4. 0x7ffe0000 הנה מה שכתב מחבר הגיליון בפוסט בנוגע למאפייני המבנה והשדה המתאים לו: המכילה נתונים המשותפים בין מצב המשתמש לגרעין (אם כי למצב המשתמש אין גישה KUSER_SHARED_DATA היא כתובת מצב המשתמש הקבועה של מבנה :כתיבה אליו). למבנה יש כמה מאפיינים מעניינים:

- הכתובת שלו קבועה והיא נמצאת בכל גרסאות Windows מאז שהוצגה
- כתובת מצב המשתמש שלה זהה במצב 32 סיביות ו-64 סיביות
- כל הקיזוים והגדלים קבועים לחלוטין, ושדות חדשים מתווספים או מתווספים רק במקום שטח ריפוד שאינו בשימוש.

לפיכך, תוכנית זו תעבוד ב-Windows 2000 של 32 סיביות וב-Windows 10 של 64 סיביות ללא קומפילציה מחדש.

קוד ++C/C

```
bool check_kuser_shared_data_structure()
{
    unsigned char b = *(unsigned char*)0x7ffe02d4;
    return ((b & 0x01) || (b & 0x02));
}
```

הקלות

עבור דגל PEB!BeingDebugged

ל-0. ניתן לעשות זאת על ידי הזרקת DLL. אם אתה משתמש ב-OllyDbg או ב-x32/64dbg כמפקח ניפוי BeingDebugged הגדר את הדגל (https://github.com/x64dbg/ScyllaHide) באגים, תוכל לבחור תוספים שונים נגד ניפוי באגים כגון

```
#ifndef _WIN64
PPEB pPeb = (PPEB)__readfsdword(0x30);
#else
PPEB pPeb = (PPEB)__readgsqword(0x60);
#endif // _WIN64
pPeb->BeingDebugged = 0;
```

עבור NtGlobalFlag

ל-0. ניתן לעשות זאת על ידי הזרקת DLL. אם אתה משתמש ב-OllyDbg או ב-x32/64dbg כמתקן ניפוי באגים, תוכל NtGlobalFlag הגדר את (https://github.com/x64dbg/ScyllaHide) לבחור תוספים שונים של Anti-Debug כגון

```
#ifndef _WIN64
PPEB pPeb = (PPEB)__readfsdword(0x30);
*(PDWORD)((PBYTE)pPeb + 0x68) = 0;
#else
PPEB pPeb = (PPEB)__readgsqword(0x60);
*(PDWORD)((PBYTE)pPeb + 0xBC) = 0;
#endif // _WIN64
```

עבור דגלי ערימה:

ל-0. ניתן לעשות זאת על ידי הזרקת DLL. אם אתה משתמש ב-ForceFlags, ואת ערך HEAP_GROWABLE ל-Flags הגדר את ערך OllyDbgScyllaHide או ב-x32/64dbg כמפקח ניפוי באגים, תוכל לבחור תוספים שונים של Anti-Debug כגון (https://github.com/x64dbg/ScyllaHide).

```

#ifndef _WIN64
PPEB pPeb = (PPEB)__readfsdword(0x30);
PVOID pHeapBase = !m_bIsWow64
    ? (PVOID)(*(PDWORD_PTR)((PBYTE)pPeb + 0x18))
    : (PVOID)(*(PDWORD_PTR)((PBYTE)pPeb + 0x1030));
DWORD dwHeapFlagsOffset = IsWindowsVistaOrGreater()
    ? 0x40
    : 0x0C;
DWORD dwHeapForceFlagsOffset = IsWindowsVistaOrGreater()
    ? 0x44
    : 0x10;
#else
PPEB pPeb = (PPEB)__readgsqword(0x60);
PVOID pHeapBase = (PVOID)(*(PDWORD_PTR)((PBYTE)pPeb + 0x30));
DWORD dwHeapFlagsOffset = IsWindowsVistaOrGreater()
    ? 0x70
    : 0x14;
DWORD dwHeapForceFlagsOffset = IsWindowsVistaOrGreater()
    ? 0x74
    : 0x18;
#endif // _WIN64

*(PDWORD)((PBYTE)pHeapBase + dwHeapFlagsOffset) = HEAP_GROWABLE;
*(PDWORD)((PBYTE)pHeapBase + dwHeapForceFlagsOffset) = 0;

```

להגנה על ערימה:

ותיקון ה-heap לאחר kernel32!HeapAlloc() ידני של 12 בתים עבור 32 סיביות ו-20 בתים בסביבת 64 סיביות לאחר ה-heap. חיבור הקצאתו.

```

#ifndef _WIN64
SIZE_T nBytesToPatch = 12;
#else
SIZE_T nBytesToPatch = 20;
#endif // _WIN64

SIZE_T nDwordsToPatch = nBytesToPatch / sizeof(DWORD);
PVOID pHeapEnd = (PBYTE)HeapEntry.lpData + HeapEntry.cbData;
for (SIZE_T offset = 0; offset < nDwordsToPatch; offset++)
    *((PDWORD)pHeapEnd + offset) = 0;

```

עבור KUSER_SHARED_DATA:

כאן `kdcom.dll` לפתרון אפשרי, אנו בדקו את הקישור שבו הטכניקה מתוארת (עם הבעיה עבור TitanHide) וגם טיוטת קוד לתיקון (<https://gist.github.com/anonymous/b5024c25634fc36e699cd9d041224531>).

(<https://anti-debug.checkpoint.com>) חזרה לעמוד הראשי

שתף את זה:

 (<http://www.reddit.com/submit?url=https://anti-debug.checkpoint.com/techniques/debug-flags.html&title=Anti-Debug:%20Debug%20Flags%20-%20Evasion%20Techniques>) 
<http://news.ycombinator.com/submitlink?u=https://anti-debug.checkpoint.com/techniques/debug-flags.html&t=Anti-Debug:%20Debug%20Flags%20-%20Evasion%20Techniques>  (https://twitter.com/intent/tweet?via=_CPRResearch_&url=https://anti-debug.checkpoint.com/techniques/debug-flags.html&text=Anti-Debug:%20Debug%20Flags%20-%20Evasion%20Techniques)  (<https://www.linkedin.com/shareArticle?>

mini=true&url=https://anti-debug.checkpoint.com/techniques/debug-flags.html&title=Anti-Debug:%20Debug%20Flags)